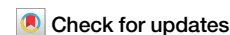


<https://doi.org/10.1038/s41534-026-01218-x>

Quantum cryptanalysis of SPN ciphers with known plaintext

Uroosa Kiran¹ ✉, Aneela Safdar², Saif Ullah Khalid³, Hammad Tanveer Butt⁴ & Zunera Jalil¹

Key recovery attack on SPN ciphers such as AES is NP hard in complexity. Previous studies have estimated the quantum resources required for implementing Grover's key search for AES, but these are largely theoretical in nature. The key challenges to implement Grover's algorithm on current quantum hardware include the number of available qubits, noise, and decoherence. However, the rapid advancement of hardware has brought us closer to practical realization for the cryptanalysis of light-weight SPN ciphers. This study presents an efficient quantum circuit implementation of Mini-AES, a 16-bit variant of AES, employing standard CNOT and Toffoli gates. To the best of our knowledge, our work provides the first complete quantum implementations of all core Mini-AES components for the quantum oracle. The effects of noise are emulated for each module of the quantum oracle and quantified. Furthermore, Grover's algorithm is applied to Mini-AES to perform successful key recovery through a known-plaintext attack, demonstrating significant amplitude amplification while using near-optimal Grover's iterations, thereby reducing the total quantum volume. The optimal Grover's iterations have been obtained empirically as well as validated with finite-shot analysis. Notably, the application of Grover's algorithm reveals key clustering behavior in Mini-AES, which has been corroborated through classical simulation.

Many NP-hard problems^{1,2} are shown to run stupendously faster using Quantum computers, which require exponential time on classical computers if the problem size is increased. Some quantum algorithms^{3–6} show practical advantage over classical ones, when big data or large and complex models are processed. This has significant implications across various domains, including information and cybersecurity, optimization, artificial intelligence, computational sciences, material science, drug discovery, and secure communications^{7–10}. Therefore, many companies and research institutions around the world are actively advancing the development of quantum hardware^{11–13} and algorithms^{14–17}. Their collective efforts are focused on unlocking the actual potential of quantum computing by creating systems capable of addressing real-world challenges across various domains^{18–20}.

One of the critical areas where quantum computing has a profound impact is cryptanalysis²¹. The security of many modern cryptographic systems relies on computational hardness (NP-hard) of mathematical primitives that classical computers struggle to break. However, quantum algorithms challenge these assumptions, posing potential threats to widely used encryption schemes^{1,22–25}. This has led to extensive research in quantum cryptanalysis to assess the vulnerabilities of classical cryptosystems

against quantum attacks. Researchers are particularly interested in how quantum speedup affect both asymmetric and symmetric cryptography, pushing the need for cryptographic agility^{26–32}.

The two significant quantum algorithms in cryptanalysis are Shor's algorithm and Grover's algorithm^{1,2} respectively. Shor's algorithm efficiently factors large integers and computes discrete logarithms; breaking number theoretic RSA and ECC encryption based on these primitives^{33,34}. While in quantum key recovery attacks on symmetric ciphers, application of Grover's algorithm is offering a quadratic speedup over classical brute-force search and reducing the time complexity from $O(N)$ to $O(\sqrt{N})$ ³⁵. Although this time complexity is still exponential, but half the effort of the classical search. Theoretically, Grover's algorithm can attack the Advanced Encryption Standard (AES)³⁶, and discover a 256-bit key in $\sim 2^{128}$ quantum queries³⁷, whereas a classical exhaustive search requires 2^{256} queries. This reduces the effective security of AES by half.

However, quantum computers are still in the development stage and the quantum resources required^{38,39} to execute a full Grover attack on AES-128/256 are currently far beyond the practical capabilities of quantum hardware. Therefore, lightweight Substitution Permutation Network (SPN) ciphers^{40–43} become an ideal choice to implement Grover's key search, using

¹Department of Cyber Security, Air University, Islamabad, Pakistan. ²Quantum Communication Lab, Evacuee Trust Complex, Islamabad, Pakistan. ³Quantum-Ronics pvt Ltd, Evacuee Trust Complex, Islamabad, Pakistan. ⁴Quantum Computing Lab, National University of Sciences and Technology (NUST), Islamabad, Pakistan. ✉e-mail: 211884@students.au.edu.pk

Quantum SDK in a simulator. The toy versions of AES such as Simplified AES (S-AES) and Mini-AES^{44,45} use only a 16-bit key but employ all the core operations involved in block SPN ciphers such as substitution, permutation and key scheduling without being overwhelmed by the full computational burden of AES.

To exploit these lightweight cryptographic schemes using Grover's algorithm, one must implement the target cipher as a quantum circuit, treating it as an oracle or black box. This black box is then queried by Grover's algorithm to efficiently search for the secret key. The quantum implementation of S-AES and Mini-AES ciphers provides practical insights into the quantum cost in terms of qubits and gate operations required for execution in quantum hardware⁴⁶, compared to earlier theoretical results^{38,39}. Since this cost would be substantially lower than implementing standard AES, the quantum circuit of light weight ciphers can be run on a simulator and optimized for cost. This, in turn, facilitates practical security testing of classical cryptographic schemes against quantum attack, while a comparable quantum hardware is still unavailable. Furthermore, reducing complexity can enhance the feasibility of hybrid cryptanalysis approaches that combine classical and quantum techniques for evaluating cryptographic security⁴⁷.

Quantum cryptanalysis on symmetric ciphers has been done with several techniques, such as Quantum Linear and Differential attacks, exploiting Hidden Shift and application of Simon or Grover's algorithm, as revised below with a focus on Grover's attack.

Along with Quantum Differential and Linear Cryptanalysis, the classical Differential⁴⁸ and Linear cryptanalysis⁴⁹ are two fundamental techniques used to analyze the security of symmetric ciphers. The corresponding Quantum versions of Differential cryptanalysis^{50–53}, leveraged quantum minimum/maximum finding algorithms in conjunction with quantum counting. The proposed method offered a quadratic speed-up over classical differential cryptanalysis, enhancing efficiency in identifying high-probability differentials. Quantum Linear cryptanalysis^{54,55} used quantum linear key-recovery attacks and algorithms like Quantum Fourier Transform (QFT)⁵⁶ or Simon's algorithm⁵⁷ to target symmetric cryptographic primitives, enhancing the effectiveness of quantum cryptanalysis.

In Quantum Attack on Hidden Shift Problem, Simon's algorithm was applied to cryptanalyze several widely used modes of operation and CAESAR candidates⁵⁸, however it countered by replacing the common bitwise addition, with other operations, such as a modular addition. Another version that tackled modular addition proposed by Xavier et al.⁵⁹ by developing new algorithms that improved and generalized Kuperberg's algorithm for the hidden shift problem and can be applied instead of Simon when considering modular additions.

Quantum Cryptanalysis using Simon Algorithm⁵⁷ is often used as a subroutine or in combination with other algorithms in order to speed-up the search problem. Its application with Grover to attack FX construction in symmetric ciphers revealed⁶⁰ that such enhancements may not provide the expected security benefits in the quantum context. Bonnetain et al.⁶¹ used Simon in offline mode in which the attacker used only classical queries and performed quantum computations separately. This adaptation allowed for practical quantum attacks without the need for quantum queries.

Quantum Cryptanalysis using Grover Algorithm² plays a pivotal role in quantum cryptanalysis by enabling quadratic speedups in brute-force key search attacks against symmetric cryptographic primitives. In order to apply Grover, target symmetric ciphers i.e., AES, DES etc. need to be implemented and optimized as quantum oracle. Quantum implementations of the AES have been studied to evaluate their security against quantum attacks and optimize their quantum circuit representations. Researchers have explored various approaches for reducing qubit requirements and improving the efficiency of AES quantum circuits, i.e., refs. 62–64, synthesizing AES circuits with fewer qubits and lower T-depth and optimizing gate counts for practical feasibility. Langenberg et al.⁶⁵ examined cost-reduction techniques for AES quantum circuits, while Lin et al.⁶⁶ and Huinan Chen et al.⁶⁷ presented an optimized implementation improving resource efficiency. Jang

et al.⁶⁸ analyzed AES using quantum cryptanalysis, highlighting vulnerabilities under quantum adversarial attack. These works collectively contributed to advance the understanding of AES security in the quantum era and developed more efficient quantum implementations. Grover's algorithm has also been analyzed against three AES-based Authenticated Encryption with Associated Data (AEAD) schemes i.e., Rocca-S, AEGIS-128, and Tiaoxin-346⁶⁹ with their optimized resource calculations. All the aforementioned work remains "theoretical" in nature, since practical implementation on current quantum hardware is not feasible.

Grover on Toy Ciphers is applied due to the complexity of AES-128/256. Jang et al. worked on quantum implementation of various simple ciphers such as PRESENT and GIFT⁷⁰ focusing on minimizing quantum resources e.g., qubits, quantum gates, and circuit depth. Albrecht et al. worked on quantum resource estimation for KATAN cipher⁷¹ offering valuable insights into its post-quantum resilience. Other reduced ciphers acting as Grover's oracle implemented by Jang et al. are PIPO, SPECK, HIGHT, CHAM, and LEA^{72–76}. Minh et al.⁷⁷ have implemented Grover attack against the YO-YO; an 8-bit SPN block cipher. These ciphers maintain essential structural properties but operate on smaller key and block sizes, making them more practical for quantum circuit simulations.

Grover on S-AES started with Almazrooe et al.⁷⁸, who constructed quantum circuits for the main components of S-AES, including Substitution, Mix Columns, and Key Expansion, and integrated them to form a complete quantum version of S-AES. This implementation was then enhanced by Jang et al.⁷⁹, reducing the number of qubits from 72 to just 32 and gate operations i.e., CX gates from 1080 to only 144. The next quantum implementation of S-AES was done by Moiseevskiy, who presented an algorithm for optimized Grover's attack on a scaled-down S-AES⁷⁹ by considering leakage of some key bits in order to be able to run the circuit on quantum simulator. His implementation reduced the required qubits to 23, 19, 15 or even 11 as key qubits leakage increases. Latest optimized implementation of this cipher has been done by Zeguo et al.⁸⁰, who proposed a general VQAA for any type of symmetric encryption that does not require quantum implementation of the encryption procedure, which substantially reduces gate complexity.

With the exception of ref. 77 and ref. 80, the majority of the aforementioned studies of light weight ciphers concentrate on estimating the computational resources required for oracle and Grover's quantum attacks, without explicitly recovering the encryption key.

In this work, we have:

- Implemented quantum oracle for Mini-AES⁴⁵ and performed component-wise noise analysis of this oracle for NISQ (Noisy Intermediate Scale Quantum Computers).
- Implemented Grover's algorithm² to recover the encryption key by employing known plaintext attack on Mini-AES.
- Observed key clustering in Mini-AES by finding multiple keys for same plain text-cipher text pairs.
- Identified an issue in the implementation of Mini-AES in Sagemath⁸¹ and developed our own Mini-AES implementation to test for the recovered keys.
- Provided an analysis of quantum resource cost for the cryptanalysis of Mini-AES.
- Achieved an ideal and optimized number of Grover iterations backed with finite-shot analysis and found that the theoretical number^{2,38,39} of Grover iterations is not practically needed, if sufficient amplification is achieved using optimal iterations.

Results

Preliminaries

As suggested, a lot of work has been done on quantum cryptanalysis of symmetric ciphers including SPN using Grover algorithm. However, almost all of it (except refs. 77, 80) is based on theoretical estimation of quantum resources (qubits, gates) and focuses on AES and S-AES. Since our work deals with Mini-AES (a different variant of AES), before diving into its

classical and quantum implementations, the differences between AES, S-AES and Mini-AES are highlighted below:

AES is the gold standard in symmetric encryption having fixed block size of 128 bits and variable key length of 128, 192 and 256 bits. Its core operations are defined over the finite field $GF(2^8)$, providing strong non-linearity and diffusion. The SubBytes uses an S-box based on multiplicative inversion in $GF(2^8)$ followed by an affine transformation, making it highly resistant to cryptanalysis. The Shift Row and Mix Column work on a 4×4 matrix of bytes to ensure rapid diffusion of bit changes across the entire state over multiple rounds. With a minimum of 10 rounds and a complex key schedule, it is secure against all known practical cryptanalytic attacks.

S-AES shares same 16-bit block and 16-bit key size as Mini-AES but has a fundamentally different design philosophy: to preserve the mathematical structure of AES in a simplified domain. S-AES meticulously mirrors the algebraic constructions of its parent. Nibble Sub is a direct scaled-down version of the AES S-box, performing multiplicative inversion in $GF(2^4)$ followed by an affine transformation. Mix Column operation is defined as multiplication by a fixed polynomial in the ring $GF(2^4)[z]/(z^2 + 1)$, directly analogous to the real AES operation. This makes it an ideal test bed for understanding the algebraic foundations of AES.

$$S - AES_E = (A_{K_2} \circ SR \circ NS \circ A_{K_1} \circ MC \circ SR \circ NS \circ A_{K_0}) \quad (1)$$

Encryption of S-AES follows Eq. (1) as does Mini-AES, however individual components are not same i.e., Eq. (2) for S-AES Mix Columns and Table 1 for S-AES Nibble Sub.

$$\begin{bmatrix} N_0 & N_2 \\ N_1 & N_3 \end{bmatrix} = \begin{bmatrix} 1 & 4 \\ 4 & 1 \end{bmatrix} \begin{bmatrix} N_0 & N_2 \\ N_1 & N_3 \end{bmatrix} \bmod (x^4 + x + 1) \quad (2)$$

Mini-AES's primary goal is to replicate the structural flow of AES with minimal parameters. It reduces the state to a $2x^2$ matrix of nibbles (4 bits) and uses a 16-bit key over 2 rounds. Mini-AES prioritizes conceptual clarity over mathematical fidelity. Its Nibble Sub is not based on finite field inversion but is instead a static 4×4 lookup table taken from the DES algorithm. Similarly, its Mix Column is a straightforward matrix multiplication using constants from $GF(2^4)$. This design makes it excellent for tracing the execution path of encryption and decryption. Table 2 summarized key differences in aforementioned ciphers.

Table 1 | Nibble-Sub in the S-AES implementation

Input	Output	Input	Output
0000	1001	1000	0110
0001	0100	1001	0010
0010	1010	1010	0000
0011	1011	1011	0011
0100	1101	1100	1100
0101	0001	1101	1110
0110	1000	1110	1111
0111	0101	1111	0111

Table 2 | Differences of AES, S-AES and mini-AES

Cipher	Primary goal	S-Box (Nibble-Sub)	Mix column
AES	Real-world security	Uses inversion in $GF(2^8)$ followed by an affine transformation	Polynomial in $GF(2^8)[z]/(z^4 + 1)$
S-AES	Mathematical replication	Uses inversion in $GF(2^4)$ followed by an affine transformation	Polynomial in $GF(2^4)[z]/(z^2 + 1)$
Mini-AES	Structural replication	Uses the 1st row of the DES S-box (lookup table)	Matrix in $GF(2^4)$

This work focuses on Mini-AES, which has unique individual modules and these are not being implemented in quantum domain in public literature before. We have not only provided practical quantum implementation of its components for the oracle, but also performed Grover's attack on specific ciphertext-plaintext pairs and demonstrated multiple key recovery. Using this comprehensive approach, we came up with novel observations such as Key clustering and optimized iterations for Grover attack that may be extended to other SPN ciphers.

This, however, is notable regarding the scope of our work that no direct comparison between the quantum implementations or resource estimation of S-AES and Mini-AES is provided due to different operations in two major components i.e., Nibble Sub and Mix Column. Also, Mini-AES cannot simply be scaled to the level of AES.

The concise explanation of the Mini-AES core components is as follows:

The nibbles in Mini-AES are elements of the finite field $GF(2^4)$, where arithmetic operations (+, −, × etc.) are closed within the field.

A nibble $n = (n_3, n_2, n_1, n_0)$, with $n_i \in \{0, 1\}$, is represented as a polynomial in the finite field $GF(2^4)$:

$$n = n_3x^3 + n_2x^2 + n_1x + n_0. \quad (3)$$

For example, the binary value 1011_2 corresponds to the polynomial $x^3 + x + 1$.

Addition in $GF(2^4)$ is performed as modulo 2 addition using an XOR ($\oplus$). For two nibbles $n = 1011_2$ and $m = 0111_2$:

$$n \oplus m = (x^3 + x + 1) \oplus (x^2 + x + 1) = x^3 + x^2. \quad (4)$$

Since XOR is its own inverse, modulo 2 subtraction is also performed using XOR.

Multiplication in $GF(2^4)$ follows standard polynomial multiplication, but the resulting polynomial may have a degree higher than 3. To ensure the result remains in $GF(2^4)$, the product is reduced modulo an irreducible polynomial. Mini-AES uses the irreducible polynomial:

$$m(x) = x^4 + x + 1. \quad (5)$$

For example, given $n = 1011_2$ and $m = 0111_2$:

$$(x^3 + x + 1) \times (x^2 + x + 1) = x^5 + x^4 + 1. \quad (6)$$

After dividing by $m(x)$, final result of multiplication in $GF(2^4)$ is x^2 .

The Encryption process of Mini-AES is summarized as follows:

$$Mini - AES_E = (\sigma_{K_2} \circ \pi \circ \gamma \circ \sigma_{K_1} \circ \theta \circ \pi \circ \gamma \circ \sigma_{K_0}) \quad (7)$$

Here symbol $\circ$ refers to composition of functions, σ_{K_0} , σ_{K_1} , and σ_{K_2} are key additions, γ is Nibble Sub, θ is Mix Column, and π is Shift Row operations. Similarly, for decryption we have,

$$Mini - AES_D = (\sigma_{K_0} \circ \gamma^{-1} \circ \pi \circ \theta \circ \sigma_{K_1} \circ \gamma^{-1} \circ \pi \circ \sigma_{K_2}) \quad (8)$$

Here, γ^{-1} is the inverse Nibble Sub, while other operations are similar but in the reverse order. In encryption, there are four main functions: Key Addition, Nibble Sub, Row Shift, and Mix Column. Each of these steps corresponds with original AES steps and plays a key role in the encryption process.

Table 3 | Nibble-sub in the mini-AES implementation

Input	Output	Input	Output
0000	1110	1000	0011
0001	0100	1001	1010
0010	1101	1010	0110
0011	0001	1011	1100
0100	0010	1100	0101
0101	1111	1101	1001
0110	1011	1110	0000
0111	1000	1111	0111

Steps involved in the process are;

- In the Add Roundkey of the Mini-AES, a 16-bit round key is XORed with a 16-bit plaintext.
- Mini-AES performs the Nibble Sub in the 4-bit chunks using S-box lookup table shown in Table 3.
- In the Shift Row, 4-bits (b_4, b_5, b_6, b_7) are simply shifted (swapped) to become 4-bit ($b_{12}, b_{13}, b_{14}, b_{15}$).
- The Mix Column operation for Mini-AES is a multiplication with a square matrix as shown in Eq. (9).

$$\begin{bmatrix} d_0 & d_2 \\ d_1 & d_3 \end{bmatrix} = \begin{bmatrix} 3 & 2 \\ 2 & 3 \end{bmatrix} \begin{bmatrix} c_0 & c_2 \\ c_1 & c_3 \end{bmatrix} \bmod (x^4 + x + 1) \quad (9)$$

It is important to have a background of Grover algorithm in order to understand the process of key recovery. Grover algorithm provides a quantum advantage over the classical search methods. For a problem of binary size n , the search space comprises of $N = 2^n$ elements, a brute-force classical approach would require $O(N)$ or $O(2^n)$ operations to find a solution in a worst-case scenario. However, Grover's algorithm enables a quantum search in $O(\sqrt{2^n})$ operations, offering a quadratic speed-up. This efficiency extends to various applications, including route optimization, unsorted database search, and other NP-complete problems that rely on exhaustive search.

It utilizes an oracle to mark solutions within a search space. The oracle is defined through a function $f(x)$, where $f(x) = 1$ indicates a valid solution and $f(x) = 0$ otherwise. The algorithm amplifies the probability of the correct solution through well-defined number of iterations. This process is particularly effective for searching unstructured solution space, reducing time complexity to $O(2^{n/2})$.

This algorithm has two main parts; Oracle and Diffuser, which together perform the function of amplitude amplification as explained below;

An oracle is a unitary operator O that acts on the computational basis states as follows:

$$O|x\rangle = (-1)^{f(x)}|x\rangle. \quad (10)$$

This phase-shift operation enables quantum amplitude amplification, facilitating the search process. The oracle essentially marks the correct solutions by flipping their phase, which is later exploited by the diffusion operator to increase their probability in the final measurement.

The oracle is constructed using a black-box function, often implemented using reversible quantum circuits. Although the specific implementation depends on the problem domain, the general structure ensures that it operates efficiently within the constraints of quantum mechanics. In particular, the oracle does not provide explicit knowledge of the solution but only identifies correct solutions when queried by giving them a phase-flip.

In this work, oracle is the quantum circuit for Mini-AES appended with Compare-Cipher Module. Mini-AES module encodes all possible cipher

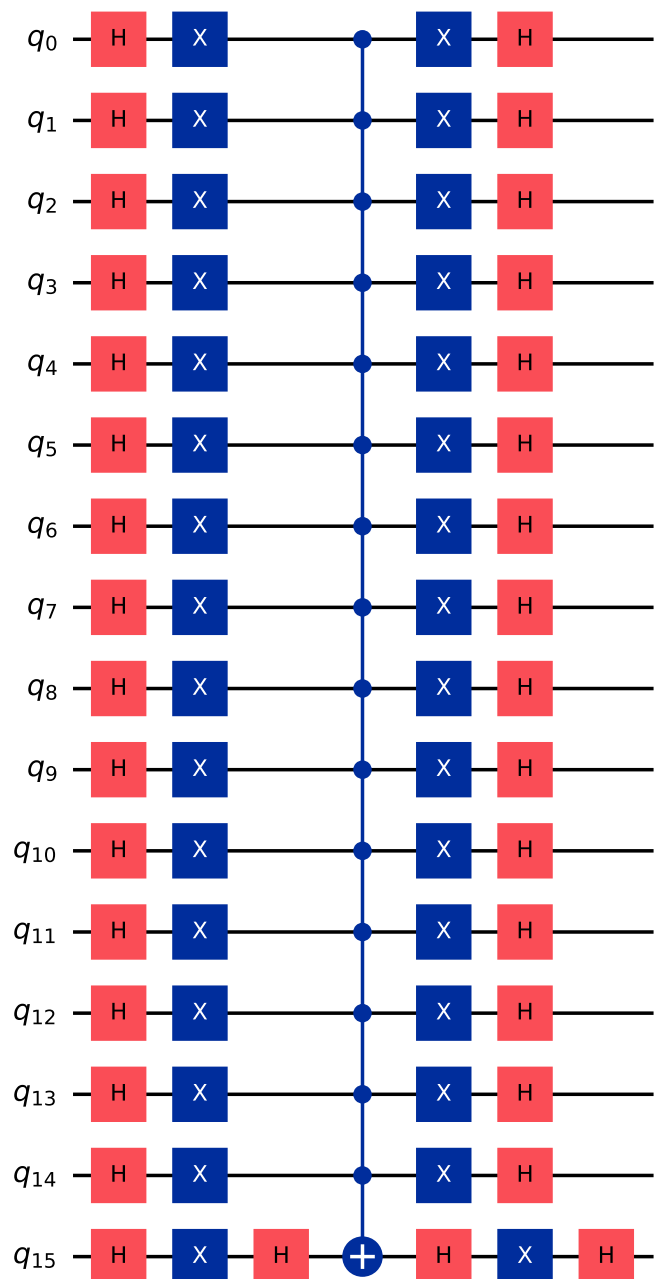


Fig. 1 | Diffuser of Grover algorithm.

texts for the given plain text using quantum parallelism and Compare-Cipher Module is used to mark the solution key(s) in Mini-AES superposition, for which the computed cipher text matches the given cipher text (see Supplementary Information section).

In Grover's algorithm, the diffuser (also known as the inversion about the mean or amplitude amplification step) plays a crucial role in increasing the probability of measuring the correct solution of a search problem.

Grover's algorithm is designed to find a marked item (or items) from an unsorted search space of N items in $O(\sqrt{N})$ queries. It uses a quantum circuit that amplifies the amplitude of the "correct" state(s) so that measuring the system is more likely to give the desired result.

The diffuser can be expressed mathematically as the operator shown in Eq. (11) and shown in Fig. 1.

$$D = 2|\psi_0\rangle\langle\psi_0| - I \quad (11)$$

Evaluation

The quantum version of Mini-AES is implemented and several test cases were generated and successfully verified with its design and our C implementation (see Supplementary Listing 1). Alternatively, a Python implementation was found in the Sagemath library⁸¹, but it generated inconsistent cipher-text. When tested thoroughly, a bug was identified in its implementation and the issue was reported⁸² which hasn't been fixed till the time of writing.

The implementation of the Mini-AES quantum circuit is done in Qiskit⁸³. The quantum resources of our implementation are summed up as in Table 4, which for general comparison, also includes the state-of-art quantum resource estimates of S-AES and AES. As both Mini-AES and S-AES are based on different modules, no direct comparison is offered, however our study presents an optimized implementation of both Nibble Sub and Mix Column, which has significantly reduced the required resources while using same 32 qubits, as of S-AES.

Noise simulation

Components of Mini-AES were also simulated under three different noise models from minimal to realistic as detailed below:-

Homogeneous Depolarizing Error ϵ_Q depicts chance of a qubit state being replaced by a maximally mixed (random) state after the corresponding gate operation. For $\epsilon = 0.005$, each qubit has a 0.5% chance of getting flipped after application of each gate. The result of simulation shown in Fig. 2 included unwanted states after measurement, however desired solution state has been amplified and dominated significantly. The Shift Row operation has contributed the least error, since only 4 gate operations occurred for this module.

In Heterogeneous Depolarizing Error ϵ_{1Q} , ϵ_{2Q} , 1 and 2 qubits gates have practically different values of ϵ . To study this affect, simulation was run under different noise models inspired by actual quantum processors offered by IBM (see Table. 5). Although these values are just snapshot and subject to vary as soon as QPUs are recalibrated but for this study purposes, these values are selected to model typical noise expected in such scenario. The values of Table 6 show complete statistics of how changing values of noise affects the frequency of states at the respective sub-module's output. For each ϵ -pair (1 and 2 qubits gates), Nibble Sub success probability ($P_{correct}$) decreases from 0.86 to 0.75 as ϵ increases; Mix Column from 0.93 to 0.90, while the $P_{correct}$ of Shift Row stayed same.

Since ϵ depicts only one kind of error, in order to run our implementation on realistic backends, a fake backend from Qiskit SDK was selected. Fake backend (device) noise model is far richer and more realistic than a single global depolarizing channel ϵ , it includes per-qubit/edge error

Table 4 | Quantum resources for Mini-AES

	Qubits	Toffoli	CX	X
AES ⁶⁷	264	16,800	57,840	830
S-AES ⁸⁰	32	120	392	59
Mini-AES (our)	32	48	176	36

rates (heterogeneous gate errors), readout/SPAM, T1/T2 relaxation, gate durations (idle errors), decomposition into basis gates, and often non-Pauli/coherent/correlated error effects. Those extra error sources (and the exact compiled circuit for that backend) produced much larger effective error than a simplistic depolarizing model previously selected. Hence success probability dropped significantly. However, it followed the same trend of Nibble Sub output being the highest affected and Shift Row the lowest, see Table 6. Following are the probable reasons of the observed trend.

- Per-output leakage probability is spread across a much larger output space (Mix Column acts on 8 qubits, 256 possible outputs; Shift Row acts on 16 qubits when combined, so individual wrong outputs are very unlikely).
- The depolarizing gates make Nibble Sub most sensitive to ϵ per gate (as of C'X gates), also observed in S-AES noisy simulations⁸⁰. Therefore Mix Column moderately sensitive due to two-qubit gates (18), and Shift Row least sensitive (only 4 swaps).

Simulating the complete Mini-AES circuit was infeasible due to its significantly increased circuit depth and qubit width, which exceeded the practical limits of the available noise-aware quantum simulation tools. Therefore, the noise characterization was performed on an individual-module basis.

Key recovery with Grover

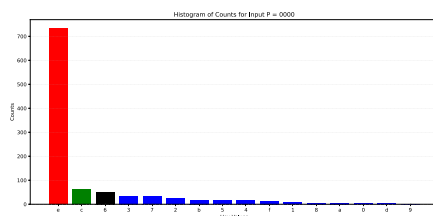
Our Mini-AES quantum implementation was evaluated using the specific plaintext and ciphertext pairs referenced in ref. 45. In one test-case, the

Table 5 | Typical 1-qubit and 2-qubit depolarizing errors (ϵ) for IBM QPUs (rounded)

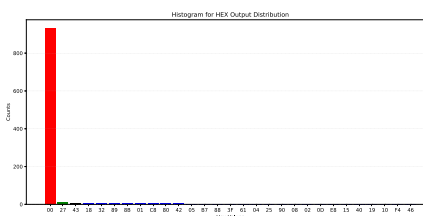
QPU name	Qubits	1Q Error (ϵ)	2Q Error (ϵ)
ibm_pittsburgh	156	0.001	0.003
ibm_kingston	156	0.002	0.004
ibm_fez	156	0.003	0.005

Table 6 | Mini-AES component noise simulation results (total counts = 1024)

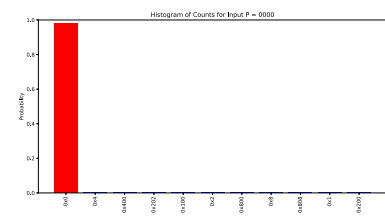
Input (ϵ)		Nibble sub		Mix column		Shift row	
ϵ_{1Q}	ϵ_{2Q}	Counts	$P_{correct}$	Counts	$P_{correct}$	Counts	$P_{correct}$
Gate counts		1Q = 3 2Q = 8 ≥ 3Q = 3		2Q = 24		2Q = 4	
0.005	0.005	733	0.71	930	0.90	1005	0.98
0.001	0.003	883	0.86	961	0.93	1013	0.98
0.002	0.004	822	0.80	950	0.92	1013	0.98
0.003	0.005	768	0.75	930	0.90	1007	0.98
FakeMontreal V2		436	0.42	470	0.45	553	0.54



(a) Nibble Sub



(b) Mix Column



(c) Shift Row

Fig. 2 | Noisy output probabilities of MiniAES components. a Nibble Sub; **b** Mix Column; **c** Shift Row: red, green, black and blue show highest to lowest values, respectively.

Fig. 3 | Grover on Mini-AES found multiple valid keys: red, green and black show three keys, rest in Blue.

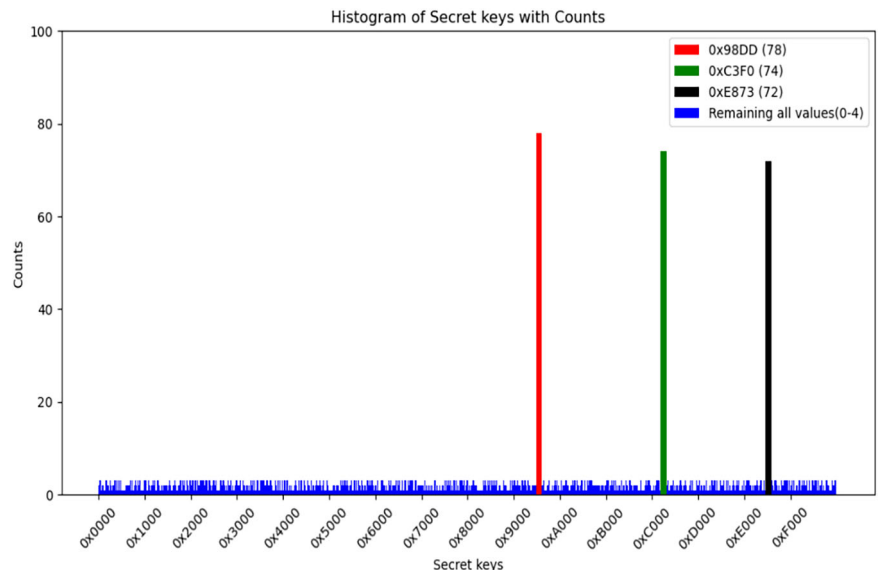


Table 7 | Count vs n for $N = 2^8$

2	4	6	8	10	12
87	270	558	799	958	1024

selected plaintext was 0x9C63, and the corresponding ciphertext was 0x72C6. The Grover attack successfully recovered the encryption key used; identified as 0xC3F0. In addition to the used key, two other potential candidate keys were also retrieved: 0x98DD and 0xE873. When evaluated using Mini-AES encryption, these additional keys were also found to be valid for this test-case. These results demonstrate the effectiveness of Grover's approach when applied to Mini-AES and are visually presented in Fig. 3.

All these multiple keys generated same plaintext-ciphertext pair, using both classical Mini-AES as well as corresponding quantum circuit implemented for Mini-AES. Sagemath library also confirmed it when its bug was fixed locally by us. These extra keys were found due to the key clustering effect inherent in the Mini-AES structure. More on Key Clustering in next subsection.

The complete Grover's algorithm for 16-qubit Mini-AES could not be executed with noise due to hardware limitations (both of simulator and available quantum processors). Therefore, we scaled down this cipher to 8-qubits, so that Grover could be run and end-to-end noise analysis can be performed. In order to create a shorter (8-bit) version of Mini-AES, we did following changes:

- Nibble Sub and Mix column remained unchanged.
- Shift Row changed to (2, 6) and (3, 7)
- For key schedule, the 8-qubit key was split into nibbles k_0 and k_1 : for example key = C6 splits to $k_0 = C$ and $k_1 = 6$ as mentioned in Algorithm 1 below.

Algorithm 1: Scaled-Down 8-bit Key Schedule. Input: Initial key bytes $w_0 = k_0$, $w_1 = k_1$

Output: Round keys $RK_1 = w_2 || w_3$, $RK_2 = w_4 || w_5$

$w_2 \leftarrow w_0 \oplus \text{NIBBLE_SUB}(w_1) \oplus \text{rcon}(1)$

$w_3 \leftarrow w_1 \oplus w_2$

$w_4 \leftarrow w_2 \oplus \text{NIBBLE_SUB}(w_3) \oplus \text{rcon}(2)$

$w_5 \leftarrow w_3 \oplus w_4$

Return: $RK_1 = w_2 || w_3$, $7D1RK_2 = w_4 || w_5$

The ideal Grover's iterations were selected with Eq. (19), keeping $N = 2^8$ and $M = 1$ and we obtained a total of 12.

Table 8 | Noise analysis on 8 qubit Mini-AES

ϵ_{1Q}	ϵ_{2Q}	Counts	P_{correct}	Counts	P_{correct}
Iteration (n)		12		3	
Ideal		1024	1	188	0.18
0.00009	0.00007	182	0.17	170	0.16
0.00012	0.0001	104	0.10	71	0.10
0.0001	0.0002	46	0.044	63	0.06
0.001	0.003	5	0.005	2	0.002
FakeMontreal V2		4	0.004	10	0.009

This 8-bit Mini-AES version was simulated under the assumed and typical ϵ values as mentioned in Table 8. The noise led to drastic reduction in the achieved counts of the solution key ($M = 1$), starting from ideal (no noise) simulation towards increasing noise scenarios (see Table 8). These results were expected as with Grover application, the depth (operations) of the circuit increases significantly, with each iteration causing additional accumulated noise.

The optimized iterations were obtained for the 8-bit Mini-AES version. Table 7 shows count under different values of n and Eq. (12) defines relationship, giving $n = 2.006$ for threshold count ≥ 100 , as optimum iterations for $N = 2^8$. We choose instead the integer value $n = 3$.

$$\text{count}(n) \approx 37.209 \cdot n^{1.421} \quad (12)$$

Using this optimized iteration number, success probability achieved against incrementing error levels is shown in Table 8. It can be seen that for most of the error levels including current NISQ device model (FakeMontreal V2), ideal Grover's iterations ($n = 12$) accumulate more noise, which cause success probability to drop more rapidly as compared to optimized Grover's iterations ($n = 3$).

With this noise analysis, we can establish that when the current quantum hardware will improve to a level that limit depolarizing errors in the range of $\epsilon \approx 0.0001$ – 0.00009 , then optimized Grover's iterations as determined in our work can be used to search for a solution key successfully.

Key clustering

To further investigate key clustering behavior, three distinct keys were selected as test cases. For each key, three corresponding plaintext-ciphertext

pairs were generated using Mini-AES Encryption process, resulting in a total of nine unique pairs. Grover's algorithm was then employed to recover the encryption keys based on these pairs. Notably, in each instance, phenomenon of key clustering was observed, wherein multiple candidate keys appeared with similar amplitude probabilities during the search process. In the first set of experiments shown in Table 9, key value "0xC0E4" was used. The plaintext-ciphertext pairs examined were 0xE0F7–0x8816, 0xFF48–0x9334 and 0x0B5D–0xA75A. By using Grover, this key was recovered in all cases and in first and second pairs, key clustering was observed, which was validated using classical implementation of Mini-AES. The selected key is shown in red, while the clustered key appears in green (second in $M = 2$) and black (third in $M = 3$).

In a second set of experiments shown in Table 10, a different key (0x4C8C) was used. The plaintext-ciphertext pairs examined were 0xA0D5–0x9BC5, 0x60B3–0x7204, and 0xE0F7–0x5F69. Grover's algorithm again successfully recovered this key in all cases. However, in the third case, key clustering was observed and an additional key was also identified.

In a third set of experiments shown in Table 11, the encryption key 0xF5E2 was used. The plaintext-ciphertext pairs examined were 0xA0D5–0xA72D, 0xBAD0–0xE7AA, and 0xFF48–0x7492. Grover's algorithm again successfully performed its required job and in the third pair, key clustering was observed again.

All these experiments demonstrated a fact that Key Clustering is very common in Mini-AES. While this does not establish the existence of same phenomenon in other SPN ciphers, but it remains an open area to explore further.

Table 9 | Key clustering in Mini-AES (key = 0xC0E4)

Plaintext	Ciphertext	Keys
0xE0F7	0x8816	0xC0E4, 0xE534, 0xA233
0xFF48	0x9334	0xC0E4
0x0B5D	0xA75A	0xC0E4, 0xC2B2, 0xC2BD

Table 10 | Key clustering in Mini-AES (key = 0x4C8C)

Plaintext	Ciphertext	Keys
0xA0D5	0x9BC5	0x4C8C
0x60B3	0x7204	0x4C8C
0xE0F7	0x5F69	0x4C8C, 0xD7C8

Table 11 | Key clustering in Mini-AES (key = 0xF5E2)

Plaintext	Ciphertext	Keys
0xA0D5	0xA72D	0xF5E2
0x0B5D	0x55E7	0xF5E2, 0xEABF
0xFF48	0x7492	0xF5E2

Table 12 | Quantum resource estimates for Grover's attack on mini-AES for different values of M

Iteration scenario	M	Iterations	X	CX	CCX	MCX	H	Swap	Circuit depth
Optimal	3	26	3128	9152	1248	1300	952	2912	4318
	2	33	3968	11,616	1584	1584	1204	3696	5480
	1	44	5288	15,488	2112	2200	1600	4928	7482
Ideal	3	116	13,928	40,832	5568	5800	4192	12,992	19,722
	2	142	17,048	49,984	6816	7100	5128	15,904	24,142
	1	201	24,128	70,752	9648	10,050	7252	22,512	34,172

Discussion

Key Clustering has been observed in Mini-AES, which, to the best of our knowledge, has not been previously reported. Key clustering refers to a situation where different encryption keys produce same cipher-text for a given plaintext. This can be viewed as a weakness in the encryption algorithm since it effectively reduces the number of unique keys and may allow an attacker with more chances to find a key that successfully decrypts the corresponding cipher-text.

The system used for implementation of Grover's algorithm on Mini-AES, using Qiskit aer simulator was Dell PowerEdge R760xs server, which includes 128GB DDR5 RAM and is powered by dual Intel Xeon Silver 4510 processors (2.4GHz, 120W, 24 cores, 16GT/s, 30MB cache).

The quantum resource estimates for Grover's iterations are summarized in Table 12. These estimates assume the use of an ideal quantum simulator for execution. A significant disparity in the number of gate operations can be observed between the optimal iteration count ($n = 44, 33, 26$) and the ideal iteration count ($n = 201, 142, 116$) for different values of M . The only notable difference in output between two approaches lies in the presence of a few non-solution keys, having average amplitudes below 1 in the optimal case. Such keys are absent in the ideal iteration results (based on Eq.(19)). Nevertheless, in both scenarios, the correct solution key can be identified with near-absolute confidence.

This paper presents a quantum implementation of the Mini-AES cipher by designing quantum circuits for all the core components including Nibble Sub, Mix Column, Shift Row, and Key scheduling, for which component-wise noise analysis is also reported. To the best of our knowledge, for the first time practically, the resulting Mini-AES quantum circuit is used in a Grover's oracle to execute a known-plaintext attack, using a single cipher-plain text pair. Our study includes a detailed analysis of the quantum resources required for the execution of this attack. Additionally, Grover's algorithm applied to perform the key recovery attack on Mini-AES, successfully retrieved 16-qubit key(s) with significant amplitude amplification. Our study also determined an optimal number of Grover iterations empirically across multiple test cases, leading to a substantial reduction in resource consumption compared to full iteration counts. This was verified through finite-shot analysis. We also carried out an end-to-end noise analysis of Grover's iterations and demonstrated an error threshold, below which the optimum number of Grover's iterations would successfully recover the secret key. Notably, our experiments also revealed the phenomenon of key clustering during the Grover attack, where multiple valid solution keys are found for a single cipher-plain text pair.

This study has been conducted as a pilot research project to establish the baselines, using idealized and noisy simulators. Our future work will focus on optimizing the proposed attack for execution on NISQ devices, under realistic hardware constraints, when noise limits become better than the baseline established in our study. Particular attention will be given to reducing quantum resource requirements, such as minimizing qubits, gate count and circuit depth, using an optimized number of Grover iterations, demonstrated in our work.

Further exploration will involve scenarios where partial key information is available. In such cases, the attack can be customized to leverage known key bits, thereby improving efficiency and reducing quantum overhead. A comparative analysis of these methods will help assess their

feasibility and effectiveness in attacking Mini-AES, ultimately contributing to the broader understanding and development of quantum cryptanalysis strategies for SPN ciphers.

Methods

Quantum implementation of Mini-AES

In order to implement the oracle of Grover's algorithm, the Mini-AES encryption process comprising of five components (i.e., Key Addition, Nibble Sub, Shift Row, Mix Column and Key Schedule) is translated into quantum implementation. Overall 32 qubits have been used, 16 for plaintext input and other 16 for the Round keys. The quantum circuit of each module is explained in the sections below.

The Key Addition (σ_{K_i}) operation involves *XORing* a 16-bit round key with a 16-bit plaintext. In a quantum circuit, this operation is efficiently implemented using 16 CNOT gates shown as Algorithm 2.

Algorithm 2: Quantum circuit for Key Addition. Input: 16-qubit Block $P(p_0, p_1, \dots, p_{15})$ and 16-qubit Block $K(k_0, k_1, \dots, k_{15})$,

Output: 16-qubit Block $K \oplus P$

for $i = 0$ to 15

CNOT (k_i, p_i)

return $P(p_0, p_1, \dots, p_{15})$

We generated the Algebraic Normal Form (ANF) equations of Mini-AES's Nibble Sub (γ). The standard function is given as,

$$f(x) = \sum_{y \in \mathbb{F}_{2^n}} f_y x^y \quad (13)$$

Here, x^y is algebraic integer that can be represented in terms of binary input bits and f_y are the coefficients of computed ANF, substituting the values of polynomials i.e., {14, 10, 3, 6, 12, 7, 10, 8, 13, 3, 6, 5, 10, 2, 10, 0} bits, the ANF equations are given as,

$$\gamma_3 = x_1 \oplus x_0 x_2 \oplus x_3 \oplus x_0 x_3 \oplus x_0 x_1 x_3. \quad (14)$$

$$\gamma_2 = 1 \oplus x_0 \oplus x_1 \oplus x_0 x_1 \oplus x_0 x_2 \oplus x_1 x_2 \oplus x_0 x_3 \oplus x_1 x_3 \oplus x_2 x_3 \oplus x_0 x_2 x_3 \oplus x_1 x_2 x_3. \quad (15)$$

$$\gamma_1 = 1 \oplus x_0 x_1 \oplus x_2 \oplus x_0 x_2 \oplus x_3 \oplus x_1 x_3 \oplus x_0 x_1 x_3. \quad (16)$$

$$\gamma_0 = 1 \oplus x_0 \oplus x_2 \oplus x_1 x_2 \oplus x_0 x_1 x_2 \oplus x_3 \oplus x_2 x_3 \oplus x_1 x_2 x_3. \quad (17)$$

$\{x_0, x_1, x_2, x_3\}$ are the input bits for the Nibble Sub operation, and $\{\gamma_3, \gamma_2, \gamma_1, \gamma_0\}$ are the output bits.

Here $\oplus$ represents the XOR operation and adjacent bits are AND together.

To achieve an optimized quantum implementation of Nibble-Sub, we used LIGHTER-R⁸⁴ tool box to process a 4×4 S-box. The resultant implementation of Nibble-Sub took only 4 qubits and implementing it for complete Mini-AES block size of 16 bits our implementation used 16 qubits. The schematic of Nibble-Sub implementation is shown in Fig. 4.

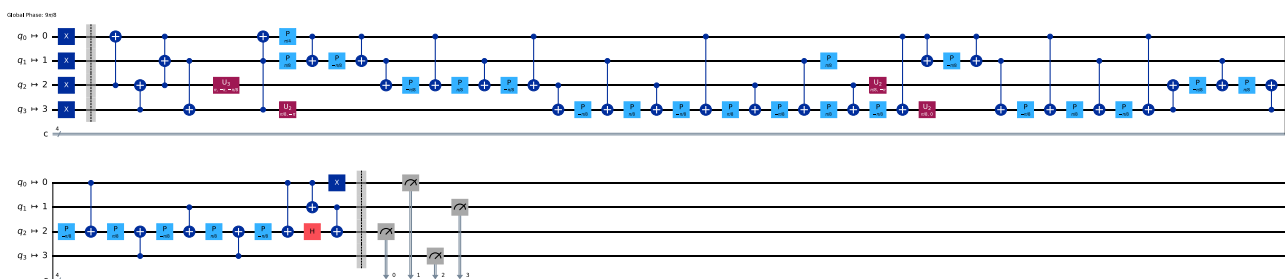


Fig. 4 | Nibble sub circuit impl using LIGHTER-R.

The ShiftRow(π) operation was implemented using quantum SWAP gates applied to the corresponding qubits, as illustrated in Fig. 5, providing a straightforward quantum realization of this transformation.

Swap (x_4, x_{12}), Swap (x_5, x_{13})

Swap (x_6, x_{14}), Swap (x_7, x_{15})

The Mix Column(θ) operation was implemented using CNOT gates. To achieve an efficient Mix Column transformation, CNOT gates were applied within an 8-qubit register. The quantum circuit for the Mix Column operation is presented in Algorithm 3.

Algorithm 3: Quantum circuit for Mix Column. Input: 8-qubit Block $X(x_0, x_1, \dots, x_7)$,

- 1: Swap (x_3, x_4) 13: CNOT (x_7, x_2)
- 2: Swap (x_2, x_5) 14: CNOT (x_7, x_3)
- 3: Swap (x_1, x_6) 15: CNOT (x_2, x_7)
- 4: Swap (x_0, x_7) 16: CNOT (x_4, x_5)
- 5: CNOT (x_3, x_0) 17: CNOT (x_1, x_7)
- 6: CNOT (x_4, x_0) 18: CNOT (x_3, x_4)
- 7: CNOT (x_4, x_3) 19: CNOT (x_1, x_6)
- 8: CNOT (x_5, x_0) 20: CNOT (x_2, x_4)
- 9: CNOT (x_5, x_1) 21: CNOT (x_0, x_6)
- 10: CNOT (x_7, x_0) 22: CNOT (x_0, x_5)
- 11: CNOT (x_6, x_1) 23: Swap (x_0, x_2)
- 12: CNOT (x_6, x_2) 24: Swap (x_5, x_7)

For an optimized implementation, PLU decomposition of multiplication matrix in Eq. (9) was utilized, enabling the use of the first 8 qubits initially and then followed by the next 8 qubits. The quantum gates for the Mix Column operation, as depicted in Fig. 6, show the optimized Mix Column circuit. Following the first barrier in the circuit, the gates corresponding to the upper matrix of the Mix Column operation were applied. Subsequently, after the second barrier, the gates representing the lower matrix were implemented. Before reaching the final barrier, two swap gates were introduced to perform the required permutation, completing the transformation as defined by the Mix Column operation.

To implement Key Schedule, We did not use auxiliary qubits, instead, the key qubits $K(k_0, k_1, \dots, k_{15})$ were utilized as follows: W_0 consisted of (k_0, k_1, k_2, k_3), W_1 consisted of (k_4, k_5, k_6, k_7), W_2 consisted of (k_8, k_9, k_{10}, k_{11}), and W_3 consisted of ($k_{12}, k_{13}, k_{14}, k_{15}$). An X-gate was applied to the k_3 qubit to form recon(1) and k_2 to form recon(2). Next, the Nibble Sub was applied to W_3 , and XOR operations were performed using CNOT gates to generate W_4 . Subsequently, CNOT gates were applied between W_1 and W_4 to produce W_5 and the process continued iteratively.

Then, in second round, recon(2) became 0010. The key schedule then updated as follows: the initial key $\{W_0, W_1, W_2, W_3\}$ was transformed into the next key $\{W_4, W_5, W_6, W_7\}$, and subsequently, $\{W_4, W_5, W_6, W_7\}$ was updated to form $\{W_8, W_9, W_{10}, W_{11}\}$, continuing the process for additional round. Figure 7, shows the 16 key qubits $K(k_0, k_1, \dots, k_{15})$, divided into four nibbles. In the last nibble $\{k_{12}, k_{13}, k_{14}, k_{15}\}$ we applied the Nibble Sub and

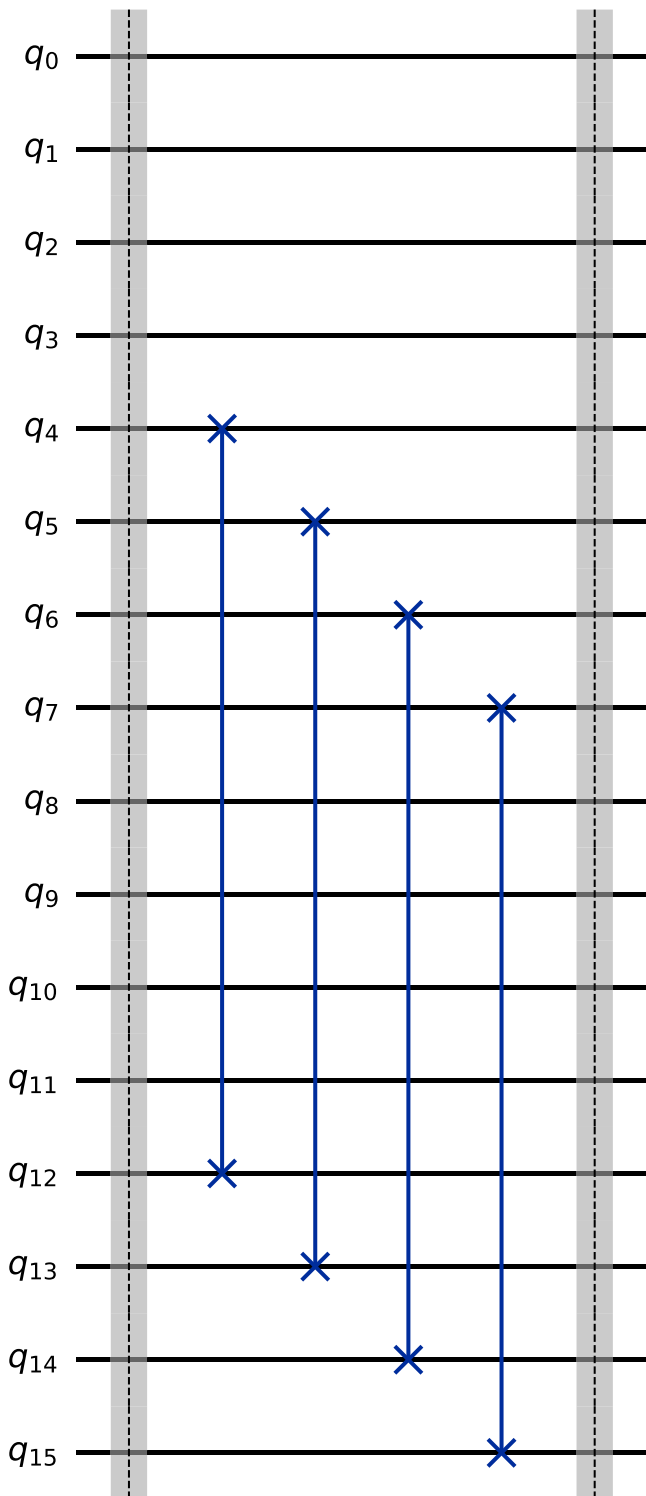


Fig. 5 | Quantum circuit for shift row.

XOR with the first nibble $\{k_0, k_1, k_2, k_3\}$ and added recon (1) using the X gate on the k_3 qubit.

To construct W_5 , XOR gates were applied between the key segments $\{k_0, k_1, k_2, k_3\}$ and $\{k_4, k_5, k_6, k_7\}$. Similarly, W_6 was generated using CNOT gates between $\{k_4, k_5, k_6, k_7\}$ and $\{k_8, k_9, k_{10}, k_{11}\}$. Before constructing W_7 , the inverse of the Nibble Sub was applied to $\{k_{12}, k_{13}, k_{14}, k_{15}\}$ to retain the original value of the last nibble. Then, XOR gates were applied between $\{k_8, k_9, k_{10}, k_{11}\}$ and the modified $\{k_{12}, k_{13}, k_{14}, k_{15}\}$ to form W_7 . Round 2 is

similar to the first, with the only difference being the use of recon(2) in place of recon(1). This is achieved by applying an X gate to k_2 .

Grover algorithm application on Mini-AES

After implementation of Mini-AES quantum circuit, the next step was to apply Grover's algorithm. The full implementation takes an example of a "known plaintext attack" with the corresponding plaintext-ciphertext pair (0x9C63, 0x72C6) mentioned in Mini-AES original paper⁴⁵. The application of Grover with Mini-AES as an oracle was done in the following steps.

1. Total 32 qubits were initialized; 16 qubits for plaintext, which were set to 0x9C63 (1001110001100011) and rest 16 qubits for the unknown key k were put into an equal superposition state (by application of Hadamard gates):

$$|\psi_0\rangle = \frac{1}{\sqrt{N}} \sum_{k=0}^{N-1} |k\rangle |1001110001100011\rangle \quad (18)$$

This initialization ensures that all possible solutions of the 16-bit key are equally probable before the search process begins, here $N = 2^{16}$. Repeat for n iterations:

2. Apply the oracle to mark the solution(s). More in-depth analysis of complete Mini-AES Oracle is given in Supplementary Information section.
 - Mini-AES circuit was used which accepted 16 plaintext qubits and 16 key qubits (in superposed state) and generated resultant 16 cipher-text qubits (also in superposed state).
 - These cipher-text qubits were compared in *Compare-Cipher* Module against cipher-text=0x72C6, which marked the solution key that generated the same cipher-text. This step flipped the phase of marked key state(s), distinguishing them from non-solutions.
 - Executed Grover diffusion operator, which amplified the probability of the marked state by increasing its amplitude.
 - Applied reverse Mini-AES in order to put plaintext qubits back to initialization.
3. Measured the output quantum state of key qubits 1024 times to obtain a solution with high probability, after n Grover's iterations.

Upon completion of the iterative process, a measurement of the quantum state resulted in a collapse onto the marked solution(s) with a high probability amplitude, see Fig. 8 for complete circuit and Fig 3 for the results of above execution.

From results, it can be seen that multiple keys were extracted by Grover's Attack on Mini-AES for this test-case. This phenomenon is known as Key Clustering and more on it later.

For this test-case, where solution space is $N = 2^{16}$ and assuming only one solution exists, i.e., $M = 1$, the no. of iterations n , required for a maximum amplification of amplitudes are found using Grover Iteration's formula Eq. (19), which become quite large 201. This is significant in terms of circuit depth and quantum volume, as $n = 201$ almost took more than a day for execution using Qiskit simulator (and would lead to accumulation of gate errors on a quantum computer). As it took quite long to run the complete iterations, an optimal number of iterations were investigated as explained in next section, which made the execution light and fast while generating solution key with reasonable level of confidence in 1024 measurements.

Selection of optimal Grover iterations for ideal case

Following the implementation of Grover's key recovery attack on Mini-AES, the next step was to estimate the number of iterations (queries) that Grover should run in order to achieve an optimal amplitude amplification within constrained resources.

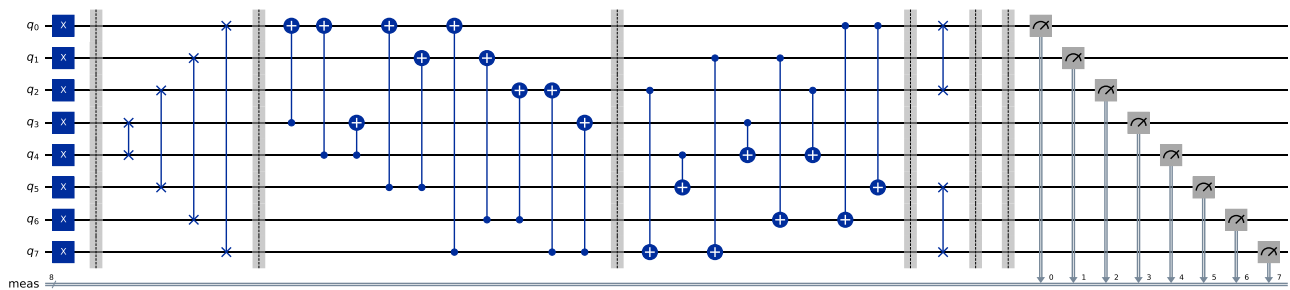
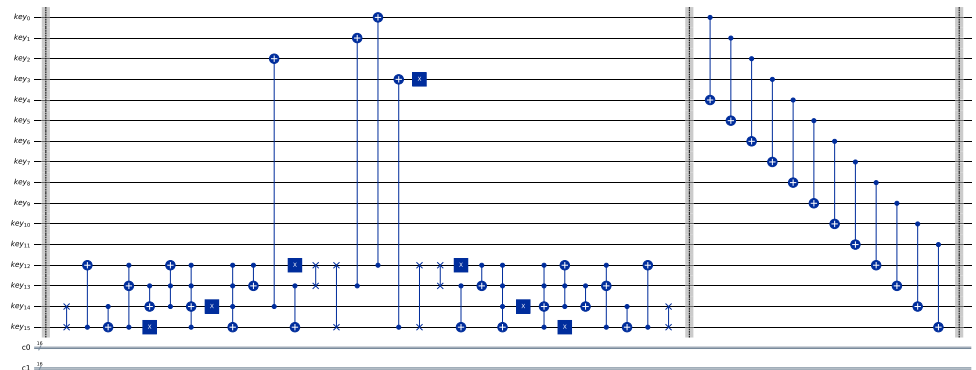


Fig. 6 | Circuit for optimized mix column.

Fig. 7 | Quantum circuit for key schedule.



The well-known formula for Grover iteration calculation is

$$n \approx \left\lceil \frac{\pi}{4} \sqrt{\frac{N}{M}} \right\rceil \quad (19)$$

where n : Number of Grover iterations or queries N : Size of the search space, for Mini-AES $N = 2^{16}M$: Number of target (marked) solutions (if not known beforehand then $M = 1$)

- For $M = 1$, iterations or queries needed are $n = 201$, and the output will only show one key with amplitude amplification of 1 ($count = 1024$).
- For $M = 2$ and $M = 3$ (assuming key clustering), $n = 142$ and $n = 116$, and the output will show two and three keys with the $count \approx 500$ and $count \approx 340$, respectively (considering total measurement counts = 1024).

In case of ideal quantum emulation on classical hardware, Quantum Mini-AES implementation with 201 Grover iterations and Qiskit's default 1024-shot measurement configuration required ~36 h to achieve near-perfect amplitude amplification. While effective for the 16-bit key case, this approach revealed significant scalability concerns, particularly regarding its feasibility for more secure, longer-key encryption systems, even on a "real" quantum computer (more depth would mean more noisy measurements).

Through systematic analysis, three critical performance dependencies were identified:

- Algorithmic complexity scaling with key size.
- Quantum resource utilization patterns.
- Execution time as a function of measurement counts.

This tripartite relationship necessitated optimization of both:

- Measurement counts (shots)
- Number of Grover query iterations (n)

Initial attempts to modify the measurement counts (shots) revealed an implementation constraint: deviations from the default 1024-shot configuration caused Qiskit simulator (aer) execution to halt while showing near-zero backend resource utilization. Community-sourced solutions from Qiskit Slack channels proved ineffective for our 32-qubit Mini-AES architecture, prompting us to instead focus on optimizing the query iterations alone.

To reduce both runtime and resource consumption while maintaining reliable key recovery, an iteration-reduction strategy was implemented for Grover's algorithm. This approach was validated through systematic data collection, where the frequency of the target key, here referred to as ($count$) in 1024 measurements, was evaluated for varying number of Grover's iterations; referred to as n for 15 different case-scenarios (10 for $M = 1$, 3 for $M = 2$ and 2 for $M = 3$). Here, M is the number of valid keys for same ciphertext-plaintext pair. Supplementary Table 1 is generated for the detailed experimental outcomes and the average values are summarized in Table 13 for these cases.

Analyzing this table made one thing clear that for $M = 1$ (i.e., one valid key); when Grover's iterations n were increased close to 201, the initial superposition of the key qubits collapsed and converged to full amplitude amplification showing single valid key as the only solution.

It also revealed a distinct relationship between iterations n and $count$ for different values of M . One can observe a dip in $count$ for $n = 200$ in cases of multiple solutions. This is evident from the fact that the max iterations required for full amplitude amplification for these cases are much lower than 201 (theoretical bound for n is 142 and 116, respectively, if Eq. (19) is followed).

By ignoring this dip and through empirical analysis, we determined that the n and $count$ followed a power-law distribution, mathematically expressed as:

$$count(n) = a \cdot n^b \quad (20)$$

This is non-linear, which can be converted into a linear form using logarithms.

$$\ln(count) = \ln(a) + b \cdot \ln(n)$$

Fig. 8 | Quantum circuit of Grover's algorithm on Mini-AES.

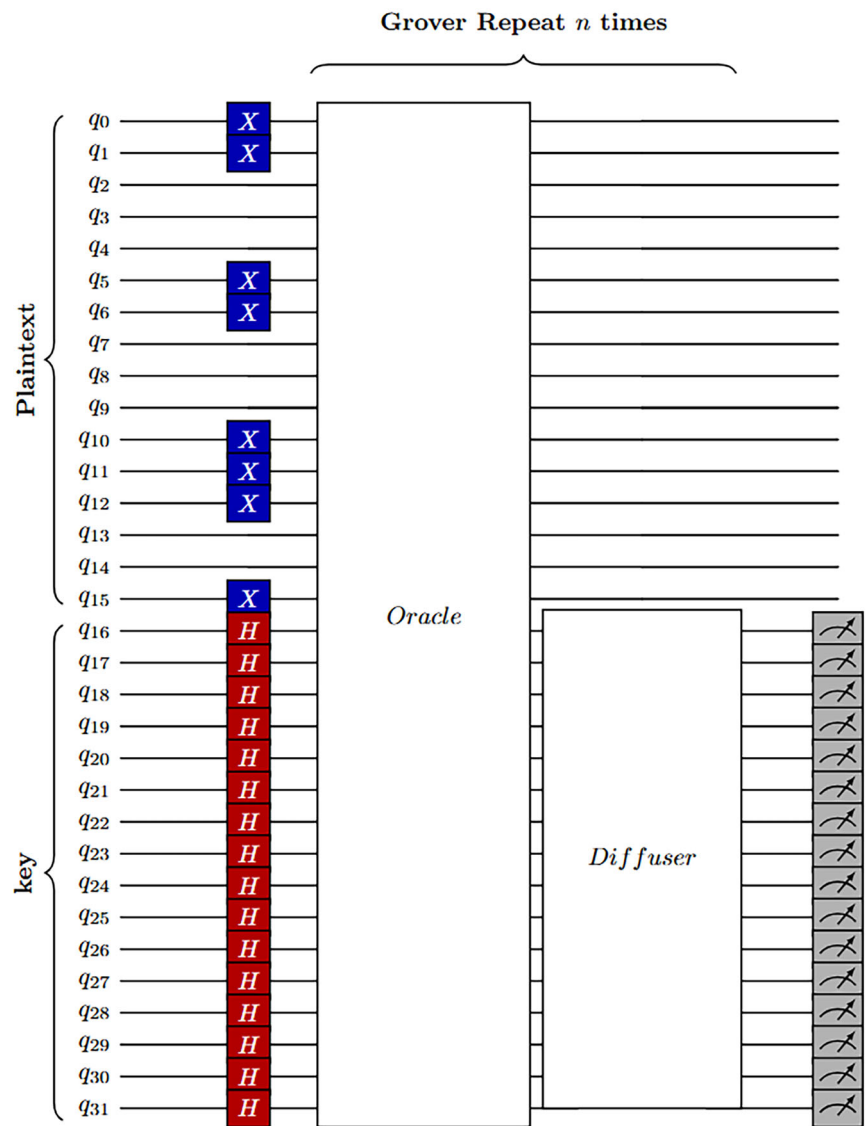


Table 13 | Mini-AES: count vs n

Iterations (n)	6	12	25	50	100	200
$M = 1$	4	11	39	151	492	1024
$M = 2$	9	16	87	284	825	624
$M = 3$	7	27	122	428	976	775

This gives a linear regression form:

$$y = c + m \cdot x$$

where

$$y = \ln(\text{count}), x = \ln(n), m = b, c = \ln(a)$$

Fitting a straight line through these points using linear regression: Solved for the slope b and intercept $\ln(a)$ using the least squares formulas:

$$b = \frac{\sum (x_i - \bar{x})(y_i - \bar{y})}{\sum (x_i - \bar{x})^2}$$

$$\ln(a) = \bar{y} - b \cdot \bar{x}$$

Taking values from Table 13,
For $M = 1$:

$$b \approx 1.547$$

$$\ln(a) \approx -1.230 \Rightarrow a \approx e^{-1.230} \approx 0.292$$

$$\text{count}(n) \approx 0.292 \cdot n^{1.547} \quad (21)$$

For $M = 2$:

$$b \approx 1.431$$

$$\ln(a) \approx -0.375 \Rightarrow a \approx e^{-0.375} \approx 0.687$$

$$\text{count}(n) \approx 0.687 \cdot n^{1.431} \quad (22)$$

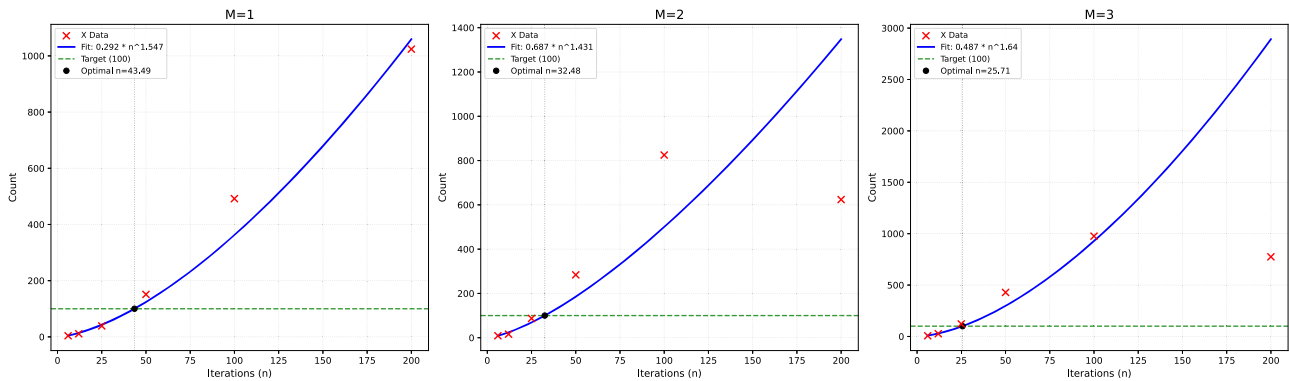


Fig. 9 | Number of iterations n vs Key frequency (*count*) in 1024 measurements.

For $M = 3$:

$$b \approx 1.640$$

$$\ln(a) \approx -0.719 \Rightarrow a \approx e^{-0.719} \approx 0.487$$

$$\text{count}(n) \approx 0.487 \cdot n^{1.640} \quad (23)$$

Also note that for $M > 1$, Eq. (22) and (23) will be valid till values of $n = 142$ and $n = 116$, after which actual simulation results showed *count* started to decrease, see Table 13.

In order to be reasonably confident that there was a significant difference in our key and the rest of the solution space, we selected *count* equal to $T = 100$ for all cases, as this made the average of the rest of the solution space < 1 . The curves in Fig. 9 were used to find the optimal n when *count* ≈ 100 . This reduced n from 201, 142 and 116 to 44, 33 and 26 for $M = 1$, $M = 2$ and $M = 3$, respectively. These down-scaled iterations (n) have reduced both time and resource consumption to an acceptable level, making the implementation suitable for scaling purposes while keeping our confidence high for significant appearance of solution(s) in the output.

In order to generalize our approach to 16 qubit SPN, we executed Grover attack on Mini-AES and S-AES for optimum iterations $n = 44$, $M = 1$ and recovered the correct key. To be precise, for $n = 40$, both S-AES and Mini-AES generated *count* ≈ 100 for the correct key. As in both cases optimum iteration for $M = 1$ successfully differentiated and recovered solution key, we can say with certain confidence that this relationship can be generalized for all 16-bit SPN ciphers.

Finite-shot analysis of the proposed Grover optimization

Grover's success probability cannot be observed directly and must be estimated from a finite number of measurement shots, therefore the resulting estimates inherently carry binomial sampling error. To formally validate the optimized iteration count, we quantify this finite-shot uncertainty and analyze its impact on the observed success probability.

Let the search space size be N , with M marked (target) states. Define the angle θ by

$$\sin^2 \theta = \frac{M}{N}, \quad 0 < \theta \leq \frac{\pi}{2}$$

After k Grover iterations, the total success probability is

$$P_{\text{tot}}(k) = \sin^2((2k+1)\theta)$$

If the marked states are symmetric, the probability to observe a *specific* marked key in one shot is

$$p_{\text{key}}(k) = \frac{P_{\text{tot}}(k)}{M}$$

When executing the oracle + diffusion circuit for S measurement shots, the number of times we observe a particular marked key C follows a Binomial distribution:

$$C \sim \text{Binomial}(S, p_{\text{key}}(k))$$

with expectation

$$\mathbb{E}[C] = S \cdot p_{\text{key}}(k) = S \cdot \frac{\sin^2((2k+1)\theta)}{M}$$

As we have chosen a detection threshold T (e.g., $T = 100$). The *expected-count rule* is

$$S \cdot \frac{\sin^2((2k+1)\theta)}{M} \geq T \quad (24)$$

For finite-shot confidence, we can also apply Hoeffding's inequality. Let $\mu = \mathbb{E}[C]$, and define

$$\delta = 1 - \frac{T}{\mu}$$

Then:

$$\Pr(C \leq T) = \Pr(C \leq (1 - \delta)\mu) \leq \exp\left(-\frac{\delta^2 \mu}{2}\right).$$

Solving $\exp(-\delta^2 \mu/2) \leq \beta$ enforces a desired false-negative probability β .

We can also demand that the true key's expected count be γ -times larger than that of a typical non-solution:

$$\frac{S \cdot |\sin^2((2k+1)\theta)/M|}{S \cdot [(1 - \sin^2((2k+1)\theta))/(N - M)]} \geq \gamma \Rightarrow \frac{P_{\text{tot}}(k)}{1 - P_{\text{tot}}(k)} \geq \gamma \cdot \frac{M}{N - M}$$

In the small- θ regime (when $M \ll N$), we can approximate $\theta \approx \sqrt{M/N}$ and, for moderate k ,

$$\sin((2k+1)\theta) \approx (2k+1)\theta$$

Then:

$$p_{\text{key}}(k) \approx \frac{(2k+1)^2 \theta^2}{M} \approx \frac{(2k+1)^2}{N}$$

and from Eq. (24) we obtain approximately

$$2k+1 \gtrsim \sqrt{\frac{TN}{S}}, \text{ so } k \approx \frac{1}{2} \left(\sqrt{\frac{TN}{S}} - 1 \right). \quad (25)$$

Below is the Worked example for $N = 2^8$ and 2^{16} :
Keeping $M = 1$, $S = 1024$ shots, target $T = 100$ and

$$\theta = \arcsin \left(\sqrt{\frac{1}{N}} \right)$$

For any $N = 2^{16}$

- Solving the exact condition, Eq. (24) numerically for integer k gives $k = 41$, since $\mathbb{E}[C] \approx 103.9$. The approximate formula Eq. (25) yields $k \approx 39.5$, in good agreement. This supports our empirically chosen $n = 44$. For any $N = 2^8$
- The approximate formula Eq. (25) yields $k = 2$, which is the optimized n .

Algorithmic recipe is summarized below:

- Fix N and (conservatively) M (e.g., $M = 1$).
- Choose S (shots) and threshold T .
- Compute $\theta = \arcsin \sqrt{M/N}$.
- Find the smallest integer k satisfying $S \cdot \sin^2((2k+1)\theta)/M \geq T$. Optionally adjust k using the Hoeffding bound for confidence.
- If considering noise, replace $\sin^2((2k+1)\theta)/M$ by the noisy success probability $\tilde{p}_{\text{key}}(k)$, measured by a noisy simulator, and re-solve for k .

This derivation justifies reduced/optimized iteration counts in both Mini-AES and S-AES context and generalizes to any small N under shot-limited regimes.

Data availability

The data used in this paper for optimization of Grover's iterations is available in Supplementary Table 1.

Code availability

Code is available at: <https://github.com/rebal87/Grover-attack-on-Mini-AES.git>.

Received: 23 July 2025; Accepted: 3 March 2026;

Published online: 01 April 2026

References

- Shor, P. W. Polynomial-time algorithms for prime factorization and discrete logarithms on a quantum computer. *SIAM J. Comput.* **26**, 1484–1509 (1997).
- Grover, L. K. A fast quantum mechanical algorithm for database search. *Proc. Annu. ACM Symp. Theory Comput.* **28**, 212–219 (1996).
- Peruzzo, A. et al. A variational eigenvalue solver on a photonic quantum processor. *Nat. Commun.* **5**, 4213 (2014).
- Farhi, E., Goldstone, J. & Gutmann, S. A quantum approximate optimization algorithm. Preprint at <https://arxiv.org/abs/1411.4028> (2014).
- McArdle, S. et al. Quantum computational chemistry. *Rev. Mod. Phys.* **92**, 015003 (2020).
- Biamonte, J. et al. Quantum machine learning. *Nature* **549**, 195–202 (2017).
- Ajagekar, A. & You, F. Quantum computing for energy systems optimization: challenges and opportunities. *Energy* **179**, 76–89 (2019).
- Bauer, B. et al. Quantum algorithms for quantum chemistry and quantum materials science. *Chem. Rev.* **120**, 12685–12717 (2020).
- Cao, Y., Romero, J. & Aspuru-Guzik, A. Potential of quantum computing for drug discovery. *IBM J. Res. Dev.* **62**, 6–1 (2018).
- Long, G. et al. Quantum secure direct communication and deterministic secure quantum communication. *Front. Phys. China* **2**, 251–272 (2007).
- Arute, F. et al. Quantum supremacy using a programmable superconducting processor. *Nature* **574**, 505–510 (2019).
- Zhong, H. S. et al. Quantum computational advantage using photons. *Science* **370**, 1460–1463 (2020).
- Krinner, S. et al. Realizing repeated quantum error correction in a distance-three surface code. *Nature* **605**, 669–674 (2022).
- Harrow, A. W., Hassidim, A. & Lloyd, S. Quantum algorithm for linear systems of equations. *Phys. Rev. Lett.* **103**, 150502 (2009).
- Childs, A. M. & van Dam, W. Quantum algorithms for algebraic problems. *Rev. Mod. Phys.* **82**, 1–52 (2010).
- Lloyd, S., Mohseni, M. & Rebentrost, P. Quantum principal component analysis. *Nat. Phys.* **10**, 631–633 (2014).
- Liang, J. & Shen, S. Quantum algorithms for the generalized eigenvalue problem. *Quantum Inf. Process.* **20**, 233 (2021).
- Berek, A. Quantum computing applications in real-world scenarios. *Nat. Commun.* **11**, 1482 (2020).
- Telecom, B. Quantum computing in telecommunication—a survey. *Math.* **11**, 3423 (2023).
- Brough, Z. Commercial applications of quantum computing. *EPJ Quantum Technol.* **8**, 1 (2021).
- Swenson, C. *Modern Cryptanalysis: Techniques for Advanced Code Breaking* (John Wiley & Sons, 2008).
- Tom, J. J. et al. Quantum computers and algorithms: a threat to classical cryptographic systems. *Int. J. Eng. Adv. Technol.* **12**, 25–38 (2023).
- Bhaskar, B., Damian, P. B. & Jaffe, J. D. Quantum computing and its impact on modern cryptography. *J. Cryptol.* **33**, 993–1030 (2020).
- Childs, A. M., Kothari, R. & Somma, R. Quantum algorithms for fixed points and applications to quantum cryptography. *Quantum Inf. Comput.* **18**, 703–733 (2018).
- Benjamin, S. C., Branham, M. F. & Nielsen, M. A. Quantum computing in modern cryptography: a survey. *Nat. Rev. Phys.* **1**, 54–65 (2018).
- Macaulay, T. & Henderson, R. *Cryptographic Agility in Practice* (Infosec Global, 2019).
- Sun, H. W., Wei, C. Y., Cai, B. B. et al. Improved BV-based quantum attack on block ciphers. *Quantum Inf. Process.* **22**, 9 (2023).
- Sun, H. W. et al. Quantum attacks on type-1 generalized Feistel schemes. *Adv. Quantum Technol.* **6**, 2300155 (2023).
- Sun, H. W. et al. Quantum attacks on beyond-birthday-bound macs. *Phys. A* **625**, 129047 (2023).
- Sun, H.-W. et al. Quantum key-recovery attacks on permutation-based pseudorandom functions. *IEEE Internet Things J.* **12**, 35692–35704 (2025).
- Ciampi, M., Magri, B. & Malavolta, G. Agile cryptography: a universally composable approach. *TCC* **14432**, 515–546 (2023).
- Marchesi, L., Marchesi, M. & Tonelli, R. Reviewing Crypto-Agility and Quantum Resistance in the Light of Agile Practices. In: Kruchten, P. & Gregory, P. (eds) *Agile Processes in Software Engineering and Extreme Programming – Workshops. XP XP 2022 2023*, Lecture Notes in Business Information Processing, vol 489. Cham. (Springer, 2024)
- Milanov, E. The RSA algorithm. *RSA Lab.* **1**, 11 (2009).

34. Kapoor, V., Abraham, V. S. & Singh, R. Elliptic curve cryptography. *Ubiquity* **2008**, 1–8 (2008).
35. Jozsa, R. Searching in Grover's algorithm. Preprint at <https://arxiv.org/abs/quant-ph/9901021> (1999).
36. Daemen, J. & Rijmen, V. *The Design of Rijndael: AES - The Advanced Encryption Standard* (Springer-Verlag, 2002).
37. Sarah, D. & Peter, C. On the practical cost of Grover for AES key recovery. In *Presentation at the 5th NIST PQC Standardization Conference*, Gaithersburg, MD, USA (2024).
38. Grassl, M., Langenberg, B., Roetteler, M. & Steinwandt, R. Applying Grover's algorithm to AES: Quantum resource estimates. *PQCrypto* **9606**, 29–43 (2016).
39. Fluhrer, S. *Quantum Cryptanalysis of Hash and Block Ciphers*. Report 2017/811 (Cryptology ePrint Arch, 2017).
40. Bogdanov, A. et al. PRESENT: an ultra-lightweight block cipher. *Proc. CHES*, **4727**, 450–466 (2007).
41. Gong, Z., Nikova, S. & Law, Y. W. *KLEIN: A New Family of Lightweight Block Ciphers* (RFIDSec, 2011).
42. Beaulieu, R. et al. The SIMON and SPECK lightweight block ciphers. In *ACM/EDAC/IEEE Design Automation Conference (DAC)* (IEEE, 2015).
43. Banik, A., Chakraborti, A., Mukhopadhyay, D. & Sarkar, S. GIFT: a small PRESENT. *CHES 2017* **10529**, 321–345 (2017).
44. Musa, A., Schaefer, E. & Wedig, S. A. simplified AES algorithm and its linear and differential cryptanalysis. *Cryptologia*, **27**, 148–177 (2010).
45. Phan, R. C.-W. Mini Advanced Encryption Standard (Mini-AES): a testbed for cryptanalysis students. *Cryptologia*, **26**, 4 (2002).
46. Maciejewski, F. B. et al. Design and execution of quantum circuits using tens of superconducting qubits and thousands of gates for dense Ising optimization problems. *Phys. Rev. Appl.* **22**, 044074 (2024).
47. Hosoyamada, A. & Sasaki, Y. *Cryptanalysis Against Symmetric-key Schemes with Online Classical Queries and Offline Quantum Computations*. Report 2017/977 (Cryptology ePrint Arch, 2017).
48. Biham, E. & Shamir, A. Differential cryptanalysis of DES-like cryptosystems. *J. Cryptol.* **4**, 3–72 (1991).
49. Matsui, M. Linear cryptanalysis method for DES cipher. *Proc. Eurocrypt* **765**, 386–397 (1993).
50. Zhou, Q., Lu, S., Zhang, Z. & Sun, J. Quantum differential cryptanalysis. *Quantum Inf. Process.* **14**, 2101–2109 (2015).
51. Denisenko, D. Quantum differential cryptanalysis. *J. Comput. Virol. Hack. Tech.* **18**, 3–10 (2022).
52. Kaplan, M., Leurent, G., Leverrier, A. & Naya-Plasencia, M. Quantum differential and linear cryptanalysis. *IACR Trans. Symmetric Cryptol.* **2016**, 71–94 (2016).
53. Xu, R. X., Sun, H. W., Zhang, K. J. et al. Quantum differential cryptanalysis based on Bernstein-Vazirani algorithm. *EPJ Quantum Technol.* **11**, 83 (2024).
54. Schrottenloher, A. Quantum linear key-recovery attacks using the quantum Fourier transform. *Adv. Cryptol.* **13912**, 258–291 (2023).
55. Bonnetain, X., Leurent, G., Naya-Plasencia, M. & Schrottenloher, A. *Quantum Linearization Attacks*. Report 2021/1239 (Cryptology ePrint Arch, 2021).
56. Shor, P. W. Algorithms for quantum computation: discrete logarithms and factoring. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 124–134 (IEEE, 1994).
57. Simon, D. R. On the power of quantum computation. In *Proceedings 35th Annual Symposium on Foundations of Computer Science*, 116–123 (IEEE, 1994).
58. Kaplan, M., Leurent, G., Leverrier, A. & Naya-Plasencia, M. Breaking symmetric cryptosystems using quantum period finding. *Proc. CRYPTO*, **9815**, 207–237 (2016).
59. Bonnetain, X. & Naya-Plasencia, M. *Hidden Shift Quantum Cryptanalysis and Implications*. Report 2018/432 (Cryptology ePrint Arch., 2018).
60. Leander, G. & May, A. *Grover Meets Simon - Quantumly Attacking the FX-Construction*. Report 2017/427 (Cryptology ePrint Arch., 2017).
61. Bonnetain, X., Hosoyamada, A., Naya-Plasencia, M., Sasaki, Y. & Schrottenloher, A. Quantum attacks without superposition queries: the offline Simon's algorithm. *Proc. ASIACRYPT*. **11921**, 552–583 (2019).
62. Zou, J. et al. Quantum circuit implementations of AES with fewer qubits. In *Advances in Cryptology – ASIACRYPT 2020, Lecture Notes in Computer Science*, vol. 12492, pp. 1–20. Cham (Springer, 2020).
63. Huang, Z. & Sun, S. Synthesizing quantum circuits of AES with lower T-depth and fewer qubits. In *Advances in Cryptology – ASIACRYPT 2022, Lecture Notes in Computer Science*, vol. 13793, pp. 1–20. Cham (Springer, 2022).
64. Li, Z. et al. Novel quantum circuit implementation of Advanced Encryption Standard with low costs. *Sci. China Phys. Mech. Astron.* **65**, 290311 (2022).
65. Langenberg, B., Pham, H. & Steinwandt, R. Reducing the cost of implementing the Advanced Encryption Standard as a quantum circuit. *IEEE Trans. Quantum Eng.* **1**, 1–12 (2020).
66. Lin, D. et al. Optimized quantum implementation of AES. *Quantum Inf. Process.* **22**, 352 (2023).
67. Chen, H. N. et al. Quantum circuit for implementing AES S-box with low costs. *Quantum Inf. Process.* **25**, 59 (2026).
68. Jang, K. et al. *Quantum Analysis of AES* (Cryptology ePrint Arch., 2022).
69. Mandal, S., Anand, R., Rahman, M., Sarkar, S. & Isobe, T. Implementing Grover's on AES-based AEAD schemes. *Sci. Rep.* **14**, 21105 (2024).
70. Jang, K.-B., Song, G.-J., Kim, H.-J. & Seo, H.-J. Grover on simplified AES. In *IEEE International Conference on Consumer Electronics-Asia (ICCE-Asia)* 1–4 (IEEE, 2021).
71. Albrecht, M. R., Fernandez, J. M., Schanck, J. M. & Smith, B. Grover on KATAN: quantum resource estimation. *IEEE Trans. Quantum Eng.* **3**, 1–9 (2022).
72. Jang, K. et al. Efficient implementation of PRESENT and GIFT on quantum computers. *Appl. Sci.* **11**, 4776 (2021).
73. Jang, K. et al. *Grover on GIFT* (Cryptology ePrint Arch., 2020).
74. Jang, K. et al. Grover on PIPO. *Electronics* **10**, 1194 (2021).
75. Jang, K. et al. *Grover on SPECK: Quantum Resource Estimates* (Cryptology ePrint Arch., 2020).
76. Jang, K. et al. Grover on Korean block ciphers. *Appl. Sci.* **10**, 6407 (2020).
77. Vu, M. T. et al. Demonstration of Grover's algorithm for retrieving secret keys in a basic SPN block cipher. *CTU J. Innov. Sustain. Dev.* **16**, 8–16 (2024).
78. Almazrooe, M., Abdullah, R., Samsudin, A. & Mutter, K. N. Quantum Grover attack on the Simplified-AES. In *7th International Conference on Software and Computer Application*, 1–5 (ACM, 2018).
79. Moiseevskiy, A. Quantum-enhanced symmetric cryptanalysis for S-AES. Preprint at <https://arxiv.org/abs/2304.05380> (2023).
80. Wang, Z. et al. Reducing quantum resources for attacking S-AES on quantum devices. *npj Quantum Inf.* **11**, 157 (2025).
81. Sage Developers. *SageMath, the Sage Mathematics Software System (Version 10.2)* <https://www.sagemath.org> (2023).
82. Safdar, A. Bug in miniAES implementation of sagemath. GitHub, Issue #40069 <https://github.com/sagemath/sage/issues/40069> (2025).
83. Qiskit Developers. Qiskit: An open-source framework for quantum computing. *Zenodo* <https://doi.org/10.5281/zenodo.2562111> (2019).
84. Dasu, V. A., Baksi, A., Sarkar, S. & Chattopadhyay, A. LIGHTER-R: Optimized reversible circuit implementation for S-boxes. In *IEEE International System-on-Chip Conference (SOCC)*, 260–265 (IEEE, 2019).

Author contributions

U.K. is the primary author who conceptualized the framework, conceived, implemented and conducted the experiments and wrote the manuscript. A.S., S.U.K. and H.T.B. refined the implementation, investigated the literature, analyzed the results and expanded the manuscript for noise analysis and optimum Grover iterations. HTB and ZJ supervised and guided the research and reviewed the manuscript.

Competing interests

The authors declare no competing interests.

Additional information

Supplementary information The online version contains supplementary material available at <https://doi.org/10.1038/s41534-026-01218-x>.

Correspondence and requests for materials should be addressed to Uroosa Kiran.

Reprints and permissions information is available at <http://www.nature.com/reprints>

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

© The Author(s) 2026